



Voraxia Planet Lifecycle Specification

Canonical architecture contract from planet creation to persistent player mining

Version: v0.1

Date: 29 June 2026

Prepared By: Coding Custard Studios

Project: Voraxia

Status: Living Specification

Coding Custard Studios | Voraxia Documentation Standard



VORAXIA

Document control

Field	Value
Document title	Voraxia Planet Lifecycle Specification
Version	Version 0.1
Status	Living architecture specification
Date	29 June 2026
Owner	Coding Custard Studios
Primary source format	Markdown stored in the Voraxia repository; DOCX and PDF are review snapshots.
Change authority	Changes to persistence, networking, coordinate, physical, or generator-version contracts require an explicit amendment.

Contents

1. Purpose and authority
2. Non-negotiable principles
3. Canonical units and coordinate contracts
4. Planet data model
5. Planet lifecycle overview
6. Creation and authoritative activation
7. Global deterministic generation
8. Representation by distance and activity
9. Gravity, atmosphere, and simulation-led physics
10. PCG population and world features
11. Mining, voxels, caves, and construction
12. Persistence, dormancy, and return visits
13. Multiplayer authority and replication
14. Planet and chunk state machines
15. Current implementation baseline
16. Validation and test matrix
17. Open decisions
18. Immediate next steps
19. Change-control rule

How to use this document

Read the status labels before treating a rule as implemented. The specification deliberately includes future architecture so new code can align with the intended planet lifecycle without pretending that unbuilt systems already exist.

1. Purpose and authority

This specification is the canonical technical and design contract for a Voraxia planet from creation through player arrival, mining, base construction, unloading, and return visits. It is broader than the current code by design. It separates locked decisions, validated prototypes, planned systems, and open questions so that future implementation does not accidentally turn a sketch into a network or persistence contract.

When this document conflicts with informal notes or conversation history, the reviewed and versioned document takes precedence.

Status vocabulary

Status	Meaning
Locked decision	An architectural rule new systems must respect.
Implemented and validated	Working code or a tested prototype exists.
Planned	The intended direction. It is not yet a supported gameplay feature.
Open decision	Deliberately unresolved. Code must not assume an answer.

2. Non-negotiable principles

1. Determinism first. Untouched planetary content must be reproducible from stable inputs, principally planet identity, seed, generator version, and global coordinates.
2. Server authority. The server owns authoritative runtime state, active terrain decisions, edits, structures, gameplay objects, and persistence.
3. Persistent deltas, not full worlds. Untouched terrain is regenerated. Player and event changes are compact persistent state.
4. One physical source of truth. Terrain, gravity, atmosphere, life support, flight, PCG, and hazards query a shared authoritative planet profile.
5. Global fields before local presentation. Macro fields are evaluated in global planet space so cube-face boundaries cannot create different terrain answers.
6. Simulation-led gameplay. A 1.2 g planet or a hostile atmosphere must materially affect movement, ships, survival, and engineering decisions.
7. Streaming is not persistence. Unloading a region makes it dormant, not deleted.
8. PCG populates the world. Hand-authored content is reserved for meaningful landmarks, missions, factions, and curated exceptions.

Core rule

A planet is always mathematically defined, selectively represented, and persistently changed only where gameplay creates a difference from the base world.

3. Canonical units and coordinate contracts

Domain	Canonical unit	Contract
Persistent planet coordinates	metres, double precision	Never store planet-scale state in Unreal

Domain	Canonical unit	Contract
		centimetres.
Radius, terrain height, atmosphere altitude	metres	Authoritative planet data remains SI-style.
Gravity	m/s ²	Current surface baseline is SurfaceGravityMetresPerSecondSquared. 1 g gameplay baseline is 9.81 m/s ² .
Gameplay / Unreal local world	centimetres	Convert only at local presentation or local-simulation boundaries.
Long-range display	kilometres	Display format only. Never a storage unit.

Every planet has immutable PlanetId. Seed and GeneratorVersion are persistent generation contracts. A terrain chunk identity contains planet identity, cube face, quadtree level, and X/Y address. This is a stable logical address, not a mesh identifier.

4. Planet data model

4.1 Definition asset

A UVoraxiaPlanetDefinition describes durable design-time identity and physical baseline. Current fields include permanent planet identity, display name, seed, generator version, radius, maximum terrain elevation, maximum editable terrain depth, and surface gravity.

4.2 Replicated runtime state

At runtime, the server validates the definition and creates compact FVoraxiaPlanetRuntimeState. Clients reconstruct deterministic base content from this state. Vertex buffers, Dynamic Mesh objects, and untouched voxel payloads are never replicated as the authoritative representation.

4.3 Planned physical profile expansion

- Atmosphere top altitude, surface pressure, and density model.
- Gas mixture / composition, temperature profile, toxicity, and corrosiveness.
- Weather capability and wind parameters.
- Versioned gravitational falloff or mass-equivalent parameters.

Status

The current gravity baseline is implemented in runtime data. Atmosphere and gravity-curve details are planned contracts, not yet gameplay systems.

5. Planet lifecycle overview

Definition authored

- > server validates and creates runtime state
- > deterministic global fields become evaluable everywhere
- > representations stream according to distance and activity
- > local terrain and voxel chunks activate near interest sources
- > players mine, build, and change persistent state
- > changed chunks save deltas or snapshots
- > distant regions become dormant
- > returning players reload base generation plus persistent changes

A planet is not fully voxelised and is not fully active. It is always globally evaluable, selectively represented, and locally materialised when interaction requires it.

6. Creation and authoritative activation

6.1 Authoring

A planet definition is created for a new world. The permanent ID is generated exactly once. Seed, radius, generator version, physical profile, planet archetype, and feature rules are selected intentionally.

6.2 Server activation

1. Validate the definition.
2. Create FVoraxiaPlanetRuntimeState.
3. Replicate compact runtime state to clients.
4. Make deterministic global fields available to local systems.
5. Begin representation and chunk streaming only when an interest source requires it.

6.3 Client activation

Clients receive runtime state and independently rebuild deterministic presentation from the approved contracts. A client never invents a different seed, generator version, terrain algorithm, gravity, or atmosphere profile.

7. Global deterministic generation

7.1 Current terrain foundation

The current macro terrain generator samples a global 3D unit direction from the planet centre and returns deterministic values including height, continentalness, and mountainness. It is intentionally independent of UObjects, mesh ownership, rendering, networking, and editor state. Global-direction sampling makes the field continuous across cube-sphere faces.

7.2 Planned planet fields

- Macro elevation, basins, continentalness, and mountain belts.
- Geology and rock strata classes.
- Cave potential and depth bands.
- Resource potential and material distributions.
- Temperature, moisture, and climate proxies.

- Biome classification, hazards, anomalies, and landmark suitability.

7.3 Generator versioning

A generator version is immutable once a planet can be saved or played. New algorithms require a new version branch. A client that does not understand a version fails closed rather than rendering invented terrain that disagrees with the server.

8. Representation by distance and activity

Layer	When used	Purpose
Far orbital representation	Far from the surface	Low-cost shell, atmosphere, clouds, broad lighting, and landmark cues. No voxel collision.
Atmospheric approach	Approaching a planet	Higher-detail terrain streams in; gravity, density, drag, and thermal queries matter increasingly.
Surface terrain representation	Landed or low-flying	Detailed terrain mesh, collision, PCG inputs, and gameplay resource/feature placement.
Local volumetric terrain	Digging, caves, deformation, construction	True 3D density/voxel chunks inside a smaller active interaction volume.

8.1 Interest sources

- Players.
- Low-flying or landed ships.
- Active mining tools and excavation machines.
- Bases and structures requiring local terrain state.
- Gameplay events, mission locations, and other server-authoritative active entities.

Streaming shape is activity-aware. A fast ship needs terrain predicted ahead of travel. An underground player needs deeper vertical coverage. A stationary base needs durable, fast-recoverable state around it. The solution is an adaptive gameplay bubble, not a single permanent player-radius conversion.

9. Gravity, atmosphere, and simulation-led physics

9.1 Gravity

Gravity is a queryable local environment value, not a fixed Unreal-world Z-axis assumption. The current surface baseline comes from the planet runtime state and points toward the planet centre. A future altitude falloff model must be stable, versioned, and consistent across server and client.

Simulation requirement

A 1.2 g planet must produce approximately $1.2 \times 9.81 \text{ m/s}^2$ at its defined surface baseline. This affects character movement, falling, rover traction, braking, landing load, hover thrust, take-off ability, fuel cost, and payload planning.

9.2 Flight and propulsion

Ship systems query local gravity and atmosphere. Hovering requires upward force greater than local mass multiplied by local gravity. Aerodynamic lift, drag, control surfaces, engine performance, and re-entry behavior depend on atmospheric conditions, not a universal Earth-like default.

9.3 Atmosphere

Atmospheric pressure, density, composition, temperature, toxicity, corrosiveness, and weather potential feed life support, suits, combustion constraints, equipment wear, flight, braking, and atmospheric entry. These are physical gameplay inputs, not scanner flavour text.

10. PCG population and world features

10.1 Role of PCG

The deterministic planet generator decides what a world is. PCG determines how that world is populated. PCG consumes terrain and environmental fields to provide the bulk of scenery and environmental storytelling that cannot be hand-built by a two-person team.

10.2 PCG candidates

- Rocks, boulders, scree, cliff dressing, debris, and geological scatter.
- Vegetation, alien flora, ice, lava, crystals, and biome dressing.
- Ore indicators and non-critical resource dressing.
- Fauna habitats and ambient zones.
- Wreck fragments, abandoned machinery, minor camps, relay towers, and surface anomalies.
- Cave entrances, hazard dressing, small industrial and scavenger sites.

10.3 Persistent features

Major faction sites, mission locations, rare resources, large ruins, landmark wrecks, dungeons, and important settlements need stable generated identities and server-owned persistent state. PCG may dress their surroundings but does not replace their authority.

10.4 Respecting edits

PCG must not respawn decoration through player bases, tunnels, cleared areas, or persistent structures. It queries terrain edits, reserved/base volumes, ownership rules, and feature exclusions before populating a chunk.

11. Mining, voxels, caves, and construction

11.1 Base density contract

```
Base procedural density
+ macro surface height
+ geology / cave / ore fields
+ player and event edits
= authoritative solid-or-empty result
```

This contract is the hinge between visible macro terrain and a mineable, cave-capable world. It must be defined before full voxel implementation begins.

11.2 Mining

Mining targets active local density chunks. The server validates tools, range, permissions, and edit operations; it commits authoritative terrain/resource changes. Clients rebuild local mesh and collision from generated base data plus current revisions and edits.

11.3 Cave bases

A player may land, excavate a cave base, leave the planet, spend time in space, and later return. The cave, openings, tunnels, and base structures remain correct because their changes are persisted. They do not need to stay loaded in memory while no one is nearby.

11.4 Construction boundary

Terrain edits and player structures are distinct persistent layers. Terrain stores altered density/material state. Structures store placement, ownership, inventory, power, health, and gameplay state. Both use stable planet/chunk/local coordinate identities.

12. Persistence, dormancy, and return visits

12.1 Untouched chunks

An untouched chunk is regenerated from deterministic base fields. No saved mesh or voxel payload is required.

12.2 Edited chunks

The server persists differences from the base world: density edits, material edits, resource depletion, feature state, and a revision number.

12.3 Delta versus snapshot policy

Persistence form	Best use	Rule
Edit deltas	Modest terrain changes, mining cuts, small tunnels.	Compact operations replayed over deterministic base terrain.
Baked chunk snapshots	Heavily modified mining complexes and cave bases.	Store direct state when replay cost becomes too high.

A chunk may be promoted from delta replay to snapshot storage according to a documented compaction policy. Snapshots remain tied to PlanetId, generator version, chunk identity, and persistence schema version.

12.4 Dormancy

When no interest source requires a chunk, active runtime mesh, collision, and simulation may unload. Dirty state is durably committed first. Dormant does not mean erased.

12.5 Reload sequence

1. The server selects the chunk because an interest source needs it.
2. Base terrain is regenerated deterministically.
3. Saved deltas or a baked snapshot are loaded.
4. Persistent structures and gameplay entities are restored.
5. The server establishes the current chunk revision.
6. Clients receive needed state and rebuild local presentation.

13. Multiplayer authority and replication

Server owns	Clients derive locally
Planet runtime state; active chunk decisions; terrain edit validation and commit; chunk revisions; persistent structures; resource depletion; save and compaction operations.	Untouched terrain density and surface from approved contracts; visual meshes; safe deterministic PCG scenery; debug and inspection presentation.

13.1 Replication policy

Replicate compact facts, not full terrain meshes: runtime profile data, active chunk IDs and revisions, edit operations or compact edit state, snapshots where necessary, and gameplay-relevant object state near a client. Clients require a recovery path when a revision is missed or local reconstruction fails.

14. Planet and chunk state machines

14.1 Planet lifecycle

State	Meaning	Exit condition
Defined	Design-time definition exists with permanent identity.	Server validates and activates it.
Authoritative	Server runtime state exists and can replicate.	Interest source requires representation.
Represented	Far shell, atmosphere, broad visual layer available.	Players or ships approach.
Streamed	Surface or voxel chunks load around active interests.	Chunks become active or leave range.
Active	Terrain, collision, features, and local simulation are live.	Edits commit or interest leaves.
Persisted	Dirty chunk or structure changes are durably stored.	Region may become dormant.
Dormant	No active runtime representation is required.	Interest source returns.
Reloading	Base world plus persistent changes are reconstructed.	Chunks become active again.

14.2 Chunk lifecycle

Procedural-only

- > requested by interest source
- > generated / loaded
- > active
- > edited (optional)
- > persisted
- > dormant
- > regenerated plus edits on next request

15. Current implementation baseline

15.1 Implemented and validated

- VoraxiaPlanet plugin foundation and developer settings.
- Planet definition assets and compact replicated runtime state.
- Persistent identity, seed, generator version, radius, terrain limits, and surface gravity fields.
- Cube-sphere coordinate conventions and chunk addressing.
- Local Dynamic Mesh preview for individual patches, adjacent regions, and whole-planet diagnostic previews.
- Outward-facing triangle winding and seam validation across cube faces and chunks.
- Live editor preview controls.
- Deterministic macro terrain sampled from global direction.
- Macro terrain radial preview displacement with visual exaggeration.
- Debug colour modes for chunk address, terrain height, and terrain slope.

15.2 Planned, not implemented

- Actual orbital / far shell runtime representation.
- Atmosphere and gravity runtime queries.
- Physical flight, drag, lift, re-entry, and life-support effects.
- Terrain density/voxel contract and chunks.
- Terrain edits, snapshots, persistence, compaction, and recovery.
- Streamed collision and active chunk scheduler.
- Production PCG pipeline and persistent feature layer.
- Structures and base integration.

16. Validation and test matrix

Area	Required checks
Cube-sphere terrain	All face orientations, adjacent chunk seams, whole-planet visual pass.
Generator	Same seed/version/input produces same output on server and client; unsupported version fails closed.
Streaming	Correct activation/deactivation by distance and activity; no invalid movement gaps.
Voxels	Edit then unload/reload; adjacent edited-boundary test; cave/surface opening continuity.
Persistence	Mine/build, leave planet, restart or rejoin, return and verify exact result.
Multiplayer	Two clients observe same edits; stale revisions recover; unauthorised edits are rejected.
PCG	Deterministic scenery remains stable; persistent features survive; nothing spawns through bases.
Physics	Gravity direction/strength follows profile; atmosphere changes flight and life-support predictably.

17. Open decisions

- Exact density function, voxel resolution, and chunk dimensions.
- Edit-delta representation and snapshot compaction threshold.
- Gravity falloff model and practical physics limits near and inside planets.
- Atmospheric equations, altitude bands, and thermal-model fidelity.
- Planet archetype schema and feature-library format.
- Climate, hydrology, ocean, and weather simulation depth.
- PCG framework boundary: Unreal PCG, custom runtime generation, or hybrid.
- Base keep-alive rules and offline simulation policy.
- Persistence backend and save migration strategy.
- Public modding or world-authoring hooks.

18. Immediate next steps

1. Bank current macro terrain and diagnostic-preview work.
2. Refine macro terrain fields and formalise procedural terrain and feature vision.
3. Draft the physical planet-profile schema before ship, atmosphere, or gravity-system implementation.
4. Define the base density contract before voxel code.
5. Design chunk persistence and revision semantics before mining and construction systems.
6. Introduce PCG in layers, beginning with non-persistent scenery driven by stable terrain and environment fields.

19. Change-control rule

Architecture change gate

Changes to planet identity, generator versioning, coordinate contracts, physical units, chunk identity, multiplayer authority, or persistence semantics are architecture changes. They require an explicit amendment to this document and a corresponding code and versioning plan before implementation.

End of v0.1 baseline